

RISC vs. CISC: Instruction Set Design Principles & Trade-offs

ISA boundaries, microarchitectural costs,
and quantitative trade-offs

Dr. J. Burns

Topic 2 Outline

1. ISA Foundations & Complexity Distribution

- The programmer-visible hardware contract and historical constraints.

2. The CISC Architectural Model

- Register-memory operations, variable-length encodings, and microarchitectural costs.

3. The RISC Revolution, Principles & Comparative Metrics

- Load/store discipline, fixed 32-bit width, and direct architectural comparison.

4. Performance, Energy & Modern Processors

- Using performance equations, energy limits, legacy compatibility, and modern μ op cracking.

PART 1

ISA Foundations & Historical Context

What Is an Instruction Set Architecture?

- An ISA is the **programmer-visible functional specification** of a processor.
- Defines instructions, register files, addressing modes, and architectural state.
- Sits directly at the hardware/software boundary as a rigid abstraction contract.
- **Complexity Redistribution:** ISA design redistributes complexity between software (compilers) and hardware (microarchitectures).



A well-designed ISA survives decades of microarchitectural overhauls by maintaining this strict software contract while hardware implementation evolves underneath it.

Historical Constraints Shaped Early ISAs (CISC Era)

- **Expensive, Limited Memory (1960s–1970s):** Small memory capacities and relatively costly memory access placed a premium on minimizing binary program size.
- **Assembly-Centric Programming:** Programs were frequently hand-written; rich instructions helped express high-level constructs directly.
- **Microprogrammed Control:** Microcode made complex multi-cycle instruction behavior practical to implement without hardwiring every control sequence.
- **The Semantic Gap:** Early designers believed machine instructions should mirror high-level statements directly.

The CISC Philosophy

Designers prioritized memory efficiency and expressive instructions to support human assembly programmers, willingly trading hardware decoding complexity for smaller binary sizes.

PART 2

The CISC Architectural Model

The CISC Architectural Model

- **CISC:** *Complex Instruction Set Computer* (e.g., VAX, x86).
- **Variable-Length Encodings:** Instructions range from 1 to 15 bytes to maximize code density and pack small instructions tightly.
- **Register-Memory Operations:** Arithmetic instructions can accept memory operands directly without separate load/store steps (e.g., `add [rdi], rax`).
- **Complex Addressing Modes:** Evaluates scaled indices and indirect pointers in hardware directly during execution.

The Decoding Bottleneck

Variable-length instructions require additional front-end logic to determine instruction boundaries and support parallel decoding, increasing implementation complexity and decode energy compared with fixed-width instruction sets.

CISC Execution: Register-Memory Translation

CISC Instruction Example (x86-64)

```
add [rdi + 8], rax
```

Architectural View (Software)

- 1 single instruction fetched.
- Compact variable encoding.
- One architectural instruction reads a memory operand, performs the addition, and writes the result back to memory.

Hardware Decomposition (μ ops)

- 1 **Load:** Fetch contents of `[rdi+8]` into ALU.
- 2 **Add:** Sum loaded value with `rax`.
- 3 **Store:** Write result back to `[rdi+8]`.

One architectural instruction may be decomposed into multiple internal micro-operations. The exact decomposition is implementation-dependent and is not visible to software.

PART 3

The RISC Revolution & Principles

The 1980s Shift: Emergence of RISC

- **Empirical Workload Findings:** Early RISC studies observed that compiled programs relied heavily on a relatively small set of simple instructions and addressing modes.
- **Pipeline Regularity:** Irregular instruction formats and complex instruction behavior made pipelining and instruction decoding more difficult to organize.
- **Compiler Advancements:** Improving compilers made it increasingly practical to rely on software for register allocation, instruction selection, and scheduling.

The Compiler's New Role

RISC designs expose a simpler instruction set and rely more heavily on compilers for instruction selection, register allocation, and scheduling, while modern processors still perform extensive dynamic optimization in hardware.

The RISC Design Principles

- **Fixed 32-Bit Instruction Width:** Simplifies parallel instruction fetch and multi-decoder design.
- **Strict Load/Store Architecture:** ALU operations use *only* registers; memory access is restricted to explicit `ldr/str`.
- **Large Orthogonal Register File:** 31 general-purpose registers (AArch64 `x0–x30`) maintain active variables on-chip.
- **Regular Instruction Formats:** Consistent encodings and operand rules simplify fetch, decode, and pipeline organization.

The Hardware Benefit

Simpler instruction formats can reduce front-end decode complexity, leaving implementation resources available for structures such as caches, execution units, and prediction hardware.

Code Comparison: RISC vs. CISC Memory Addition

High-level statement: $c = a + b$; (operands in memory)

AArch64 RISC Assembly

<code>ldr x1, [x0, #0]</code>	load <i>a</i>
<code>ldr x2, [x0, #8]</code>	load <i>b</i>
<code>add x3, x1, x2</code>	compute sum
<code>str x3, [x0, #16]</code>	store <i>c</i>

x86-64 CISC Assembly

<code>mov rax, [rdi]</code>	load <i>a</i>
<code>add rax, [rdi + 8]</code>	load <i>b</i> + add
<code>mov [rdi + 16], rax</code>	store <i>c</i>

RISC explicitly uses 4 fixed-width instructions (16 bytes), while CISC uses 3 variable-length instructions. Both sequences perform the same programmer-visible computation, but their instruction counts and internal implementations differ.

Fetch Complexity: Concrete Byte Encodings

RISC Fixed 32-Bit Decode

- Every instruction is exactly 4 bytes.
- `add x0, x1, x2` $\implies$ `0x8B020020`
- Fixed boundaries make it straightforward to identify successive instruction words and feed multiple decoders in parallel.

CISC Variable-Length Decode

- `add eax, ebx` $\implies$ `0x01D8` (2 bytes)
- `add rax, rbx` $\implies$ `0x4801D8` (3 bytes)
- `add rax, [rbx+rcx*4+0x10]` $\implies$ 8 bytes!
- Requires additional front-end logic to identify instruction boundaries before or during parallel decode.

The Hardware Penalty

Determining variable-length instruction boundaries adds front-end circuitry and switching activity, which can increase decode complexity and energy consumption.

Legacy Compatibility and Encoding Overhead

- **x86-64 Compatibility Overhead:** x86-64 preserves decades of binary compatibility, and some modern 64-bit instructions use prefix bytes such as the 0x48 REX prefix in addition to legacy opcode structures.
- **Instruction-Boundary Detection:** Variable-length x86 instructions require the front end to determine where each instruction ends before it can be fully decoded.
- **AArch64 Execution State:** AArch64 introduced the A64 instruction set with fixed 32-bit encodings. Many ARMv8-A implementations historically supported AArch32 separately, while newer implementations may omit it.
- **Fixed-Width Benefit:** A64 instruction boundaries are known directly from the 4-byte alignment, simplifying instruction fetch and parallel decode relative to variable-length encodings.

ISA Comparison: AArch64 vs. x86-64

Design Metric	AArch64 (Modern RISC)	x86-64 (Modern CISC)
Instruction Width	Fixed 32-bit (4 bytes)	Variable (1 to 15 bytes)
ALU Operands	Commonly 3-operand (add d, s1, s2)	Many scalar integer forms are 2-operand; newer vector forms also support 3 operands
Memory Access	Load/Store only (ldr/str)	Register-Memory (add rax, [rbx])
General Registers	31 general-purpose registers, with architectural roles for sp/xzr	16 general-purpose registers in x86-64
Decode Logic	Fixed-width boundaries simplify parallel decode	Variable-length boundaries require additional decode machinery
Scheduling	Compiler scheduling plus dynamic hardware techniques	Compiler scheduling plus dynamic hardware techniques
Primary Benefit	Regular encoding and load/store simplicity	Code density and long-term software compatibility

Architectural Trade-Off

Fixed-width load/store ISAs simplify instruction boundaries and operand handling; variable-length ISAs can improve code density and preserve long-term binary compatibility. Modern implementations of both can use sophisticated microarchitectures.

PART 4

Quantitative ISA Trade-offs

RISC and CISC Performance Trade-offs

RISC-Style Effects

- May require more instructions for the same computation.
- Simpler instructions can reduce decode and control complexity.
- Regular operations can support lower CPI and efficient pipelining.

CISC-Style Effects

- May complete more work with fewer instructions.
- Individual instructions may require more internal work.
- More complex execution can increase CPI or implementation cost.

Performance

The Iron Law from Topic 1 still applies. A lower instruction count does not by itself guarantee lower execution time if CPI or clock period increases.

Energy, Thermal Limits & Performance-per-Watt

Dynamic Power Equation

$$P = C \cdot V_{dd}^2 \cdot f + P_{static}$$

- Complex decoders increase switching capacitance (C).
- Higher operating voltages can enable higher frequency, but increase dynamic power quadratically through the V_{dd}^2 term.

Performance-per-Watt

$$\frac{\text{Workload Throughput}}{\text{Watts}}$$

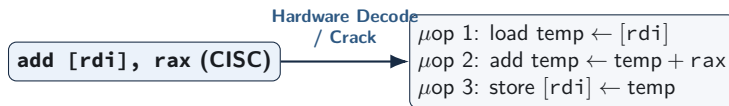
- Primary metric for mobile batteries and cloud data center racks.
- Front-end organization, instruction encoding, cache behavior, and execution width all influence energy efficiency.

The Dark Silicon Era

Power-density and thermal constraints prevent all transistors from operating at maximum activity simultaneously, making energy efficiency a central constraint in modern multicore design.

Microarchitectural Convergence: Modern x86 Internals

Modern x86 processors preserve the CISC ISA externally, while many architectural instructions are translated into simpler internal micro-operations:



- High-performance x86 cores typically execute internal micro-operations (μops) using out-of-order scheduling and other techniques also found in modern AArch64 cores.
- Translating variable-length x86 instructions into internal μops adds front-end decode overhead; fixed-width AArch64 avoids that particular boundary-detection requirement but still uses sophisticated front-end hardware.

SUMMARY & ASSESSMENT

Review and Self-Test Questions

Topic 2 Summary

- CISC tends to expose richer operations and can reduce instruction count.
- RISC tends to expose more regular operations that simplify decoding and pipelining.
- These choices can move instruction count, CPI, and clock period in different directions.
- Modern processors blur the historical RISC/CISC boundary.
- AArch64 provides a modern RISC case study for the rest of the course.

Main Point

Do not ask which philosophy is inherently faster. Ask what trade-offs the ISA creates and how those trade-offs affect the workload.

Self-Test Questions: ISA Principles & Mechanics

- ❶ **Load/Store Boundary:** Why does a strict load/store architecture simplify pipelined execution compared to an architecture permitting memory operands directly in ALU instructions?
- ❷ **Parallel Decode Efficiency:** Why do fixed 32-bit instruction boundaries simplify wide instruction fetch and decode compared with variable-length encodings?
- ❸ **Micro-operation Translation:** When an x86 processor executes `add [rdi + 8], rax`, what kinds of internal μ ops might a processor use to implement the architectural behavior?

Self-Test Questions: ISA Trade-offs & Power

- ① **ISA Trade-off:** Give one reason why a CISC-style instruction might reduce instruction count but still not reduce execution time.
- ② **Front-End Power Cost:** Why can complex instruction decoding increase front-end power use?
- ③ **Performance-per-Watt:** Why is Performance-per-Watt often more useful than raw clock frequency when comparing modern processors?

Looking Ahead

Next Lecture Preview

Our Next Topic: The Hardware/Software Interface & AArch64 Programmer's Model

- **Architectural State:** Defining the precise software-visible hardware contract: registers, processor state, stack pointer, program counter, and memory.
- **AArch64 Register File:** Examining register naming conventions ($x0-x30$ vs. $w0-w30$), the zero register (xzr), and standard ABI calling conventions.
- **Instruction Mechanics:** Establishing AArch64 syntax and programmer-visible instruction behavior before detailed arithmetic and control flow in Topic 4.